Executive Summary: Automation Portfolio

Miguel Villarreal · Finance operations → automation · Python · SQL · LLMs · .NET · México

Repository: github.com/MVillarreal98/fde-portfolio

WHAT THIS IS

Three demo projects built around problems from billing and payment operations. All data is synthetic and was written for the demos. No project has users or a production deployment, and no figure below measures business impact. The figures are test counts and measurements on synthetic data, and each project README gives the command, machine and versions behind them.

PROJECTS: PROBLEM → ROOT CAUSE → SOLUTION → EVIDENCE

Project	Problem and root cause	Solution	Evidence (synthetic data)
Reconciliation Engine (Python, pandas, DuckDB SQL)	A ledger and a settlement file disagree. Failure modes it guards against (several were found in its own v1.0.0): lookups that do not aggregate per reference, IDs read as numbers and malformed amounts dropped silently.	Text-only ingestion, money as integer cents, one-to-one join per reference with one status each, 7 rules, rejected-rows file, conservation checks, exit codes for schedulers. The same rules run in pandas and in DuckDB SQL.	171 tests, 100% statement and branch coverage on pandas 2.3.3 and 3.1.0. Parity tests cover both engines. 1M rows per side: full run in 12.7 s (pandas) or 11.1 s (SQL), a single run on one desktop machine.
LLM Support Desk (Python, Ollama, SQLite, Docker)	Free-text payment claims need decisions that follow policy. A model's answer is unreliable and can be injected.	The local LLM proposes and deterministic guardrails decide, and they can only escalate. JSON schema whitelist, keyword fallback, SQLite decision log with SQL metrics, offline eval harness.	133 tests, 99% line and branch coverage, prompt-injection tests. Eval on 23 labeled tickets: `llama3.2:1b` 13/23 category agreement against 21/23 for the keyword fallback; 9 of 10 must-escalate tickets escalated on the LLM path.
Document Layout Automator (C#, .NET 8, WPF + CLI)	Fixed-width files are built by hand, and one extra character shifts every later field. Padding without length checks causes silent truncation or overflow.	Declarative layout profiles. Lines are exactly the sum of the field widths, and an oversized value rejects its row. Per-row errors with line, field and reason. The write is fail-closed and shared by the GUI and the CLI.	38 xunit tests, including golden tests on the committed sample output. Builds with `-warnaserror` and 0 warnings.

HOW IMPACT WOULD BE MEASURED (NOT MEASURED YET)

- Reconciliation: share of references reconciled automatically, share of findings that finance confirms, and time from file arrival to a reviewed report.
- Support desk: escalation recall and over-escalation on labeled real tickets, the rule-override rate, and agreement with the final human resolution.
- Layout automator: files blocked before upload, ingestion errors at the receiving system, and time per file compared with the manual process.

SKILLS SHOWN IN THE CODE

- Data: pandas pipelines, DuckDB SQL (CTEs, window functions, `FULL OUTER JOIN`, gaps-and-islands), integer-cent money handling, conservation checks.
- Applied LLMs: local Ollama, versioned prompt, JSON output contract, deterministic guardrails, adversarial tests, offline evaluation.
- Databases: SQLite decision log and SQL metrics (`json_each`), DuckDB.
- Desktop: C# and .NET 8 (WPF), a console CLI, xunit with golden tests.
- Engineering: pytest with coverage gates, ruff, mypy, GitHub Actions (pandas 2/3 matrix, Windows .NET job), multi-stage Docker with a test stage.

BACKGROUND (CONTEXT, NOT IMPACT OF THESE TOOLS)

Billing Specialist at Central Transport (Monterrey, Nov 2023 – Jul 2025): 240+ bills of lading a day and 1,000+ invoices a month, billing accuracy 99%+ (figures from my CV). The projects were built after that role and were not used there. The synthetic data uses Mexican retail payment rails (OXXO cash payments, SPEI, cards) as its setting, with no company affiliation.

HOW IT WAS BUILT

Built with the help of AI coding tools. Each project's README documents its design decisions, how it is tested and its known limitations.